Section (1) – Intro to Python

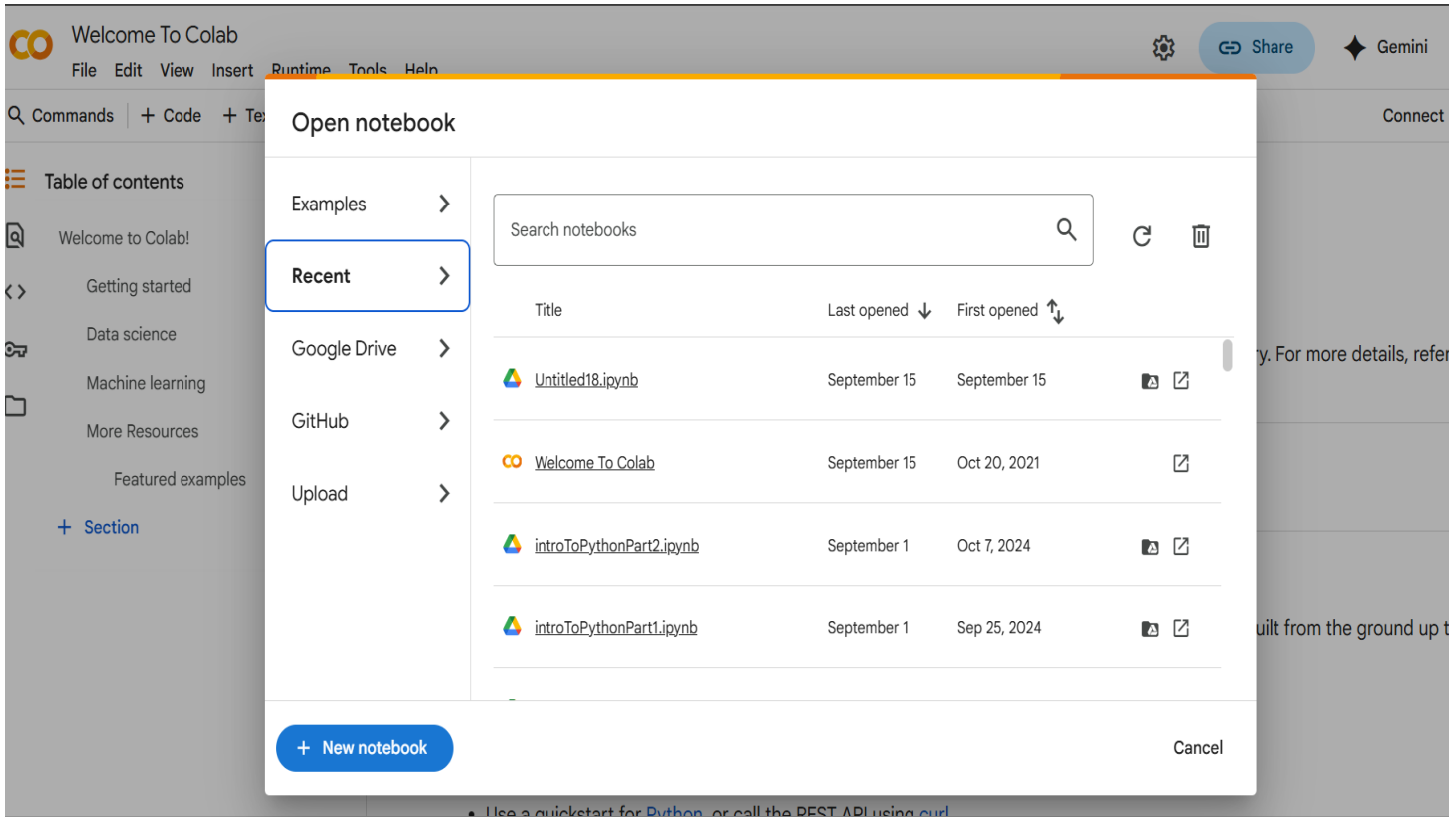
Eng. Abdulrahman ElMadkhoum

How to Install Python

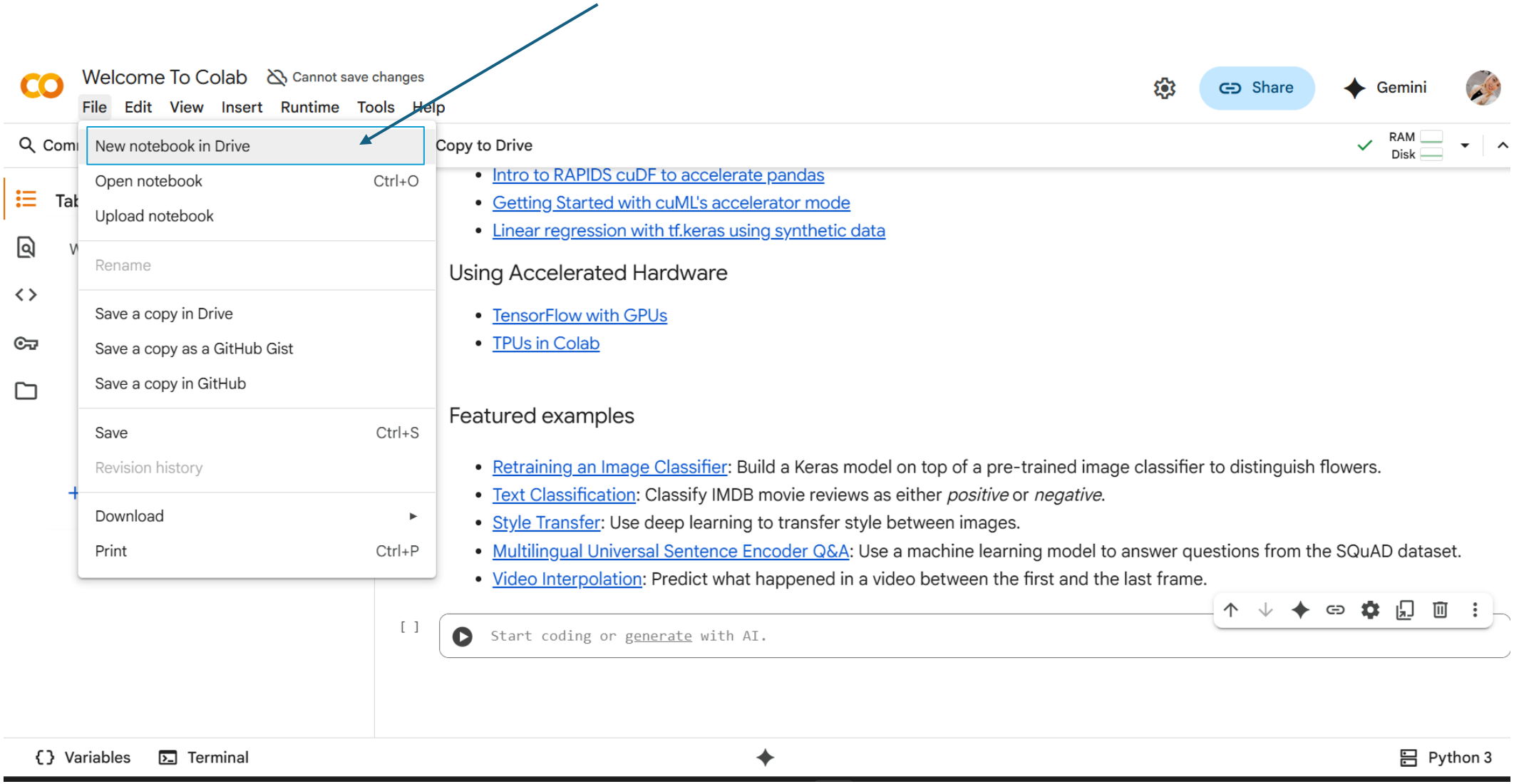
For macOs :
https://www.youtube.com/watch?v=NirAuEAblvo

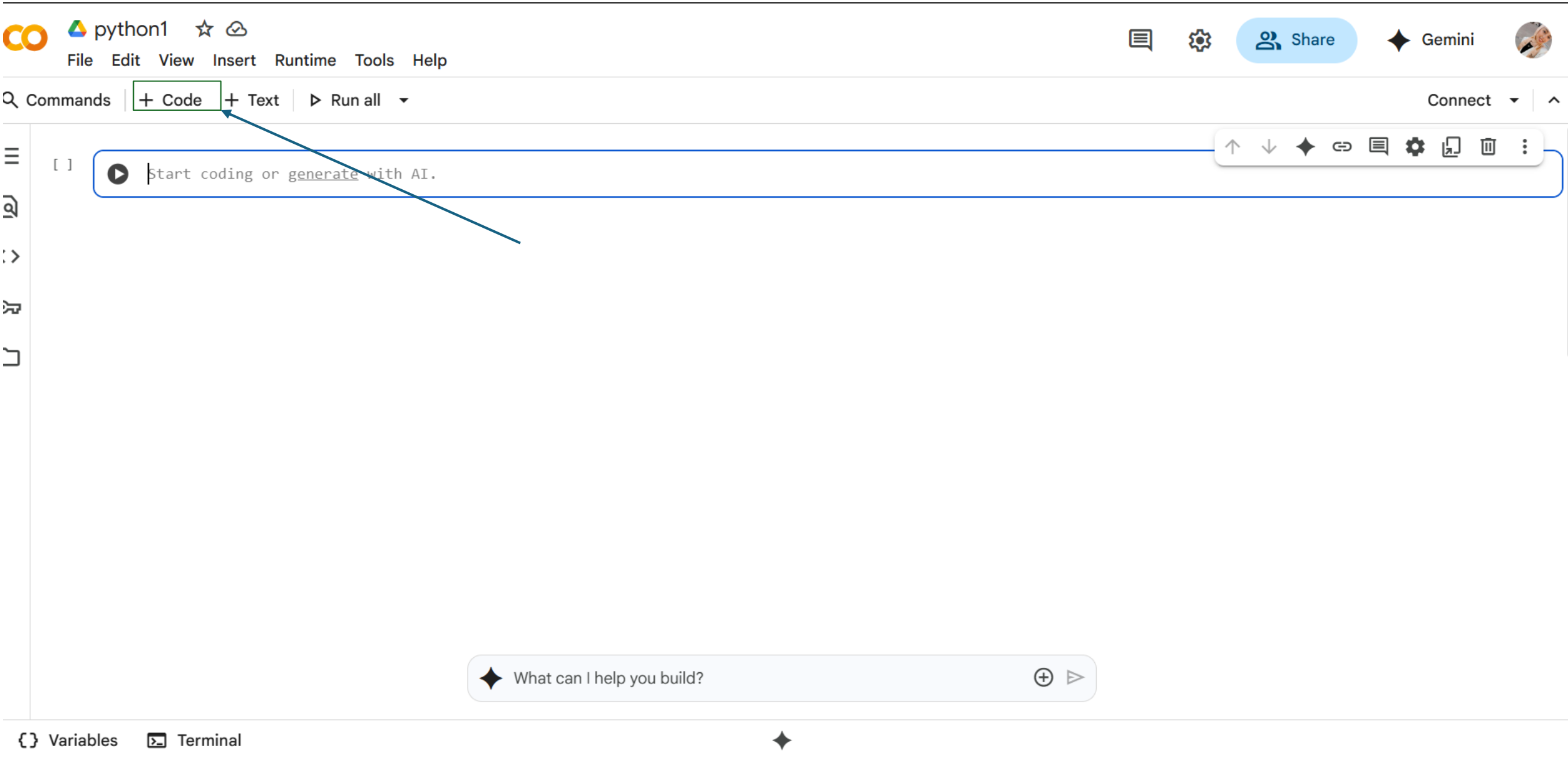
For Windows :
https://www.youtube.com/watch?v=9o4gDQvVkLU

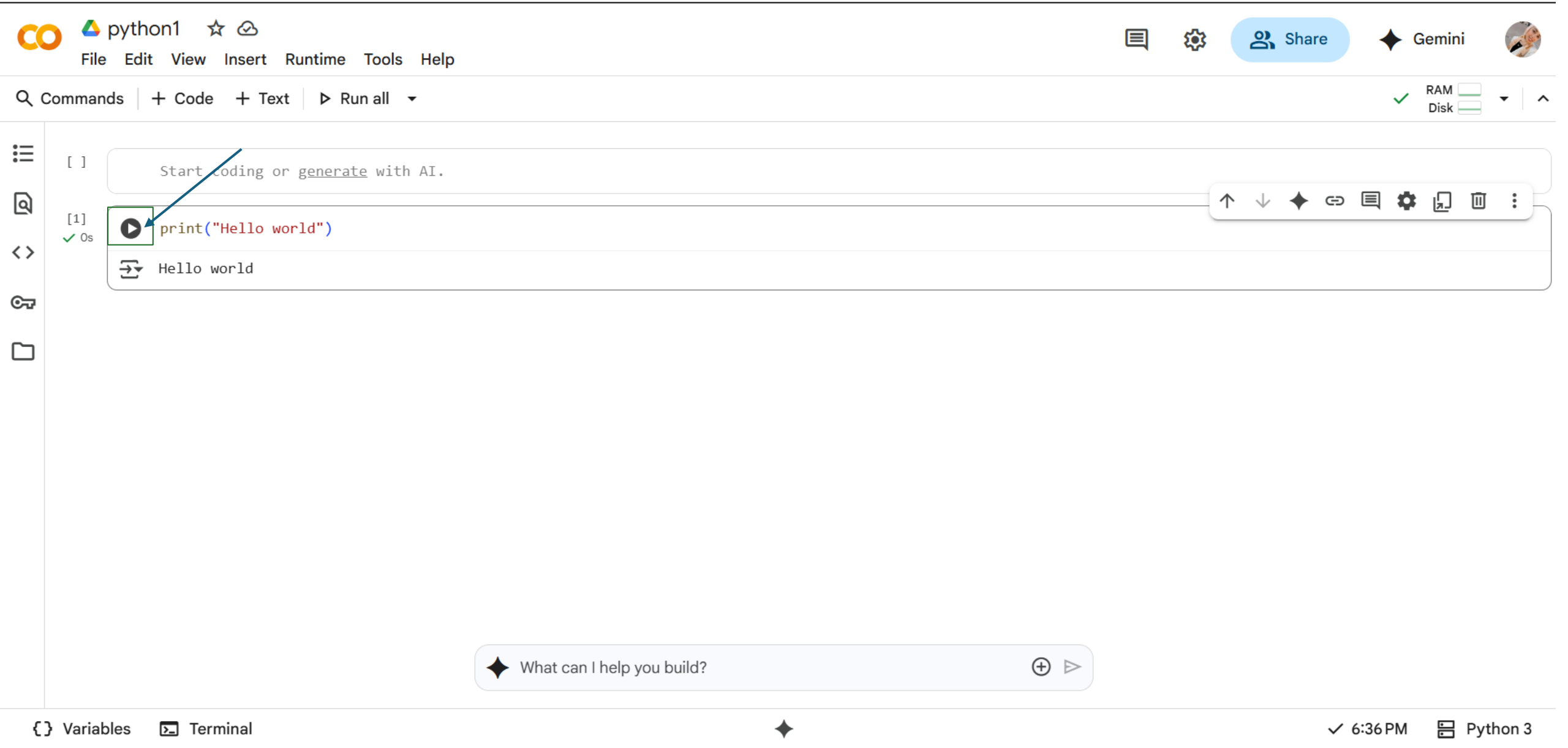
Or Without installation you can use Google Colab



- https://colab.research.google.com/


Welcome To Colab
Cannot save changes
File Edit View Insert Runtime Tools Help
Share
Gemini
New notebook in Drive
Open notebook Ctrl+O
Upload notebook
Rename
Save a copy in Drive
Save a copy as a GitHub Gist
Save a copy in GitHub
Save Ctrl+S
Revision history
Download
Print Ctrl+P
Copy to Drive
RAM
Disk
Intro to RAPIDS cuDF to accelerate pandas
Getting Started with cuML's accelerator mode
Linear regression with tf.keras using synthetic data
Using Accelerated Hardware
TensorFlow with GPUs
TPUs in Colab
Featured examples
Retraining an Image Classifier: Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
Text Classification: Classify IMDB movie reviews as either positive or negative.
Style Transfer: Use deep learning to transfer style between images.
Multilingual Universal Sentence Encoder Q&A: Use a machine learning model to answer questions from the SQuAD dataset.
Video Interpolation: Predict what happened in a video between the first and the last frame.
[ ]
Start coding or generate with AI.
Variables
Terminal
Python 3


python1
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
Connect
[ ]
Start coding or generate with AI.
What can I help you build?
Variables
Terminal


python1
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[ ]
Start coding or generate with AI.
[1]
0s
print("Hello world")
Hello world
What can I help you build?
Variables
Terminal
6:36 PM
Python 3

Python Programs

A Python program is a sequence of definitions (like assigning a value to a variable or defining a function) and commands (like printing a message) that the Python interpreter evaluates and executes.

Python code can be:

- Typed directly in the interactive shell, or
- Saved in a file (called a Python script, .py) which the interpreter reads and runs.

Python is an interpreted language, meaning the code is translated and executed line by line, unlike compiled languages (like C), where the whole program is converted into machine code before running.

Python in AI

- **Easy to read and write** – great for testing and experimenting with ideas .
- **Lots of AI tools** – supports machine learning and deep learning with libraries like TensorFlow, PyTorch, scikit-learn, pandas, and NumPy.
- **Big community** – most AI tutorials and research papers in Python.

Print

By default, Print function adds new line you can avoid this by using the parameter end = “”

[3]
✓ 0s

```
print("Hello, world!")  # Prints text inside quotes
print('Python is fun!')
```

```
Hello, world!
Python is fun!
```

[4]
✓ 0s

```
print("Hello, world!",end=" ")  # Prints text inside quotes
print('Python is fun!')
```

```
Hello, world! Python is fun!
```

Comments

```
#This is a Comment

'''
  This is a multi-line comment
  using triple single quotes.
  It can span multiple lines
  and is often used for documentation.
'''
```

Python Objects

Everything in Python is an **object**.

- Each object has a **type**, which defines:
  - The **operations** you can do with it
  - The **range of values** it can hold
- **Types of objects:**
- **Scalar (indivisible)** – single value objects
  - int → integers: -3, 5, 1000
  - float → real numbers: 3.0, 4.28, -5.5
  - bool → Boolean values: True or False
  - NoneType → special object with value None
- **Non-scalar (structured)** – objects with internal structure
  - Strings, lists, tuples, dictionaries, sets

Defining Variables in Python

```python
name = "John"
GPA = 3.4
age = 21
is_student = True
print(name,GPA,age,is_student)  # You can print multiple Values Using comma
```

```
John 3.4 21 True
```

Variable Naming Conventions

Variable names are **case-sensitive** (Age ≠ age)

Can contain **letters, numbers, underscores** (_)

Must **start with a letter or underscore**

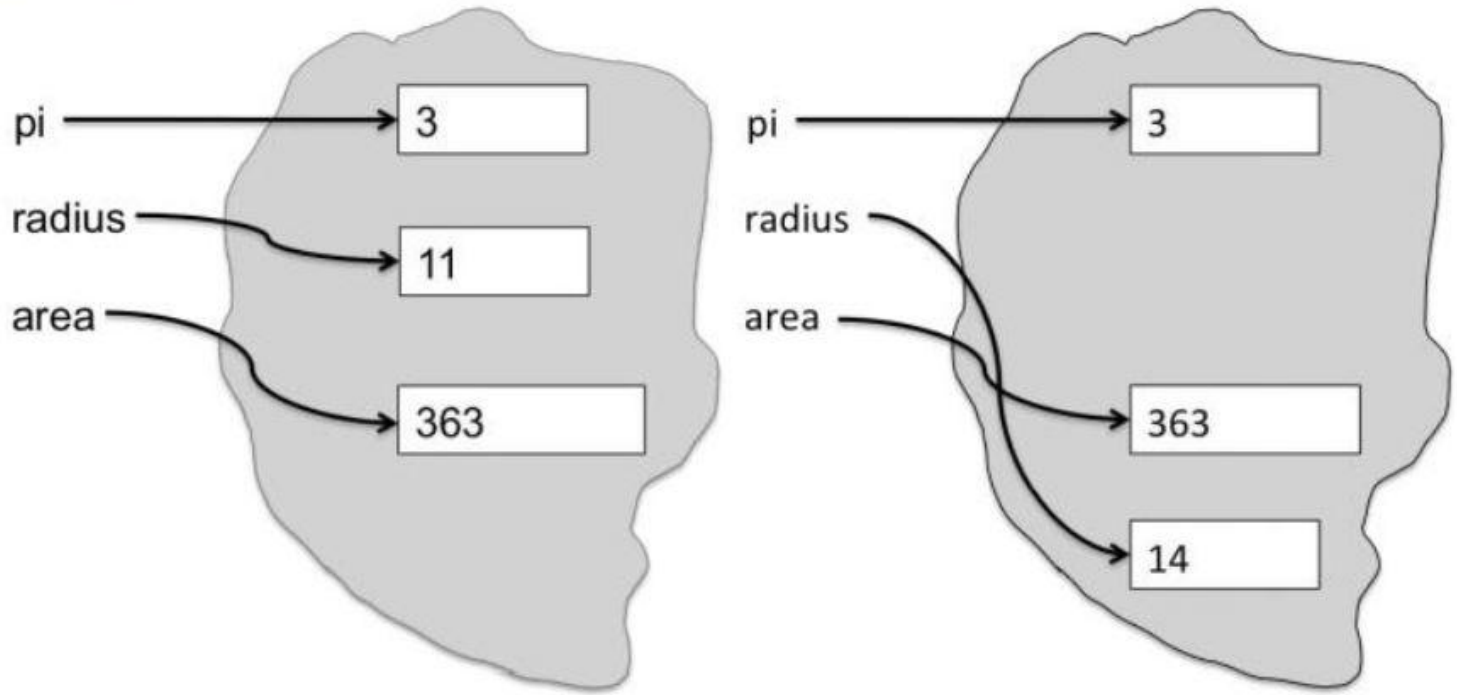
Cannot be **Python keywords** (if, for, while, etc.)

Recommended: **use descriptive names**

Variables and Assignment:

Variables provide a way to associate names to expressions which makes it easier to reuse expressions as follows:

```
pi = 3
radius = 11
area = pi * (radius**2)
radius = 14
```



As depicted in the figure, it first binds **pi** to **int** object **3** and **radius** to **int** object **11**. Then, it binds **area** to a third **int** object which is calculated using the int values of variables **pi** and **radius**. Finally, **radius** is **rebound to** another **int** object **14** which does not affect the int value to which area is bound.

You Can Examine the type of object using type function

```
x = 5
print(type(x))

y = 3.14
print(type(y))

z = "Hello"
print(type(z))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

You Can Also Convert from type to type

```
num1 = 2
num2 = 3.9
print(float(num1))
print(int(num2) )
```

```
2.0
3
```

Arithmetic Expressions

```
x = 10
y = 3

print(x + y)   # Addition
print(x - y)   # Subtraction
print(x * y)   # Multiplication
print(x % y)   # Modulus (remainder)
print(x ** y)  # Exponent (power)
print(x / y)   # Division (float result)
print(x // y)  # Floor division (truncates decimal)
```

```
13
7
30
1
1000
3.3333333333333335
3
```

1. Highest precedence

** → Exponentiation

Associativity: Right-to-left

2 ** 3 ** 2

Same as 2 ** (3 ** 2)

2. Multiplication, Division, Floor Division, Modulus

*, /, //, %

Same precedence → evaluated **left-to-right**

3. Addition and Subtraction

+, -

Same precedence → evaluated **left-to-right**

Use **parentheses** () to override precedence and make expressions clear.

Comparison (Relational) Expressions

```python
a = 5
b = 10

print(a == b)   # equality
print(a != b)   # inequality
print(a > b)    # greater
print(a < b)    # lesser
print(a >= b)   # gte
print(a <= b)   # lte
```

```
False
True
False
True
False
True
```

Logical Expressions

```python
x = True
y = False

print(x and y)  # False
print(x or y)   # True
print(not x)    # False
```

```
False
True
False
```

Multiple Assignment & Swapping

```
x, y = 2, 3

print(f"Before : x = {x} , y = {y}")

x, y = y, x

print(f"After : x = {x} , y = {y}")
```

```
Before : x = 2 , y = 3
After : x = 3 , y = 2
```

import

What is import in Python?

import is a statement used to bring external modules or libraries into your program.

A module is a file containing Python code—functions, variables, classes—that you can reuse.

Why do we use import?

Reuse existing code – no need to write everything from scratch.

Access specialized functions – e.g., math, random, date, statistics.

Organize code – modules help keep programs clean and modular.

Save time and reduce errors – reliable, tested code is already available.

```python
import math

# Square root
print("√16 =", math.sqrt(16))

# Power
print("5^3 =", math.pow(5, 3))

# Factorial
print("5! =", math.factorial(5))

# Circle area
radius = 4
print("Area =", math.pi * radius**2)
```

```
√16 = 4.0
5^3 = 125.0
5! = 120
Area = 50.26548245743669
```

Strings

- A string is a sequence of characters enclosed in single ' ' or double " " quotes.
- Strings can include letters, numbers, symbols, or spaces.

Using + with strings means concatenation

Using * means repetition

```
name = "Ahmed"
print(name * 5 )
print("Hello "+ name)
print("Hello", name)
```

```
AhmedAhmedAhmedAhmedAhmed
Hello Ahmed
Hello Ahmed
```

Input

we use Input function for taking input from users and You can print a message for user of what is required to be entered

```
name = input("Enter Your Name")
print("Hello "+name+" !")
```

```
Enter Your NameJohn
Hello John !
```

Input

- Python uses the input() function to **take input from the user**.
- It **always returns a string**, even if the user types a number.

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")

result = num1 + num2

print("Result:", result)
```

```
Enter first number: 1
Enter second number: 2
Result: 12
```

Input

- So we often convert the input using int() , float() if a number is expected

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

result = num1 + num2

print("Result:", result)
```

```
Enter first number: 1
Enter second number: 2
Result: 3
```

Taking Multiple inputs at Once

```
num1, num2 = map(int, input("Enter two numbers separated by space: ").split())

print("Result = ",num1 + num2)
```

```
Enter two numbers separated by space: 1 2
Result =  3
```

Branching

Branching allows a program to choose between different actions based on a condition.

In Python, branching is done using if, elif, and else statements.

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

```
Enter your age: 18
You are an adult
```

Branching

- Write a Python program that asks the user to **enter their GPA** (on a scale of 0.0 to 4.0) and prints the **grade** according to the following ranges:

GPA Range	Grade
3.7 – 4.0	A
3.0 – 3.69	B
2.0 – 2.99	C
1.0 – 1.99	D
0.0 – 0.99	F

Answer

```python
# Input GPA from the user
gpa = float(input("Enter your GPA (0.0 - 4.0): "))

# Determine grade based on GPA ranges
if 3.7 <= gpa <= 4.0:
    grade = "A"
elif 3.0 <= gpa < 3.7:
    grade = "B"
elif 2.0 <= gpa < 3.0:
    grade = "C"
elif 1.0 <= gpa < 2.0:
    grade = "D"
elif 0.0 <= gpa < 1.0:
    grade = "F"
else:
    grade = "Invalid GPA"

print("Your grade is:", grade)
```

```
Enter your GPA (0.0 - 4.0): 2.3
Your grade is: C
```

Empty cell

Loops : For Loop

range() generates a sequence of numbers, often used with for loops.

```
for i in range(6):  # range starts from 0 by default
    print("Number:", i)
```

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Loops : For Loop

You can specify The start of the loop

```
for i in range(1, 6):  # 1 to 5
    print("Number:", i)
```

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

For Loop

```
for i in range (6,-1,-1):
  print("Number:", i)
```

```
Number: 6
Number: 5
Number: 4
Number: 3
Number: 2
Number: 1
Number: 0
```

While Loop

```
i = 0
while(i<10):
  print(i , end=" ")
  i+=1
```

```
0 1 2 3 4 5 6 7 8 9
```

While Loop

```
i = 0
while(i<10):
  print(i , end=" ")
  i+=1
```

```
0 1 2 3 4 5 6 7 8 9
```

Question

Write a Python program that:

Asks the user how many numbers they want to enter (n).

Takes n numbers as input in a loop.

Calculates and prints:

- The **sum of all numbers**
- The **product of all numbers**
- The **average of the numbers**

Answer

```python
n = int(input("How many numbers do you want to enter? "))

total = 0
product = 1

for i in range(1, n+1):
    num = float(input(f"Enter number {i}: "))
    total += num
    product *= num

average = total / n

print("Sum:", total)
print("Product:", product)
print("Average:", average)
```

```
How many numbers do you want to enter? 4
Enter number 1: 1
Enter number 2: 2
Enter number 3: 3
Enter number 4: 4
Sum: 10.0
Product: 24.0
Average: 2.5
```

What is a Function?

A function is a block of code that performs a task and can be reused.

Syntax:

```
def function_name (parameters) :
    # code block
    return value  # optional
```

Factorial

```python
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result *= i
    return result

num = int(input("Enter a number: "))
print("Factorial of", num, "is", factorial(num))
```

```
Enter a number: 5
Factorial of 5 is 120
```

Recursion

- **Recursion** is when a function **calls itself**.
- Must have a **base case** to stop, otherwise it causes **infinite recursion**.

```python
def factorial(n):
    if n == 0 or n == 1:  # Base case
        return 1
    else:
        return n * factorial(n-1)  # Recursive call

num = int(input("Enter a number: "))
print("Factorial of", num, "is", factorial(num))
```

```
Enter a number: 5
Factorial of 5 is 120
```

Classroom code : x4dojxnd

Thank You